

Paxos Made Simple

Paxos 通俗讲解

Leslie Lamport

2001 年 11 月 1 日

摘要

如果用普通的语言来表述，Paxos 算法其实非常简单。

目录

1	引言	1
2	共识算法	1
2.1	问题	1
2.2	选定一个值	3
2.3	获知被选定的值	13
2.4	进展	14
2.5	实现	15
3	实现状态机	17

1 引言

用于实现容错分布式系统的 Paxos 算法一直被认为很难理解，这是因为它最初的表述对许多读者来说像希腊文一样晦涩 [5]。事实上，它是最简单、最显然的分布式算法之一。它的核心是一个共识算法，也就是 [5] 中的“synod”（教会）算法。下一节将说明，这个共识算法几乎不可避免地从我希望它满足的性质中推导出来（译者注：这句话有点拗口，它其实表述的意思是，我们可以自然地假设我们希望这个算法拥有某种“特性”，然后从这些特性分析，自然而然地导出 Paxos 算法流程）。最后一节会解释完整的 Paxos 算法：只要把共识算法直接应用到构建分布式系统的状态机方法上，就能得到它。状态机的做法应该已经广为人知，因为它是分布式系统理论中也许被引用最多的那篇文章 [4] 的主题。

2 共识算法

2.1 问题

假设有一组进程可以提出值。共识算法要保证：在这些被提出的值中，只会有一个值被选定。如果没有任何值被提出，那么就不应该有值被选定。如果某个值已经被选定，那么进程应该能够得知这个被选定的值。共识的安全性要求如下：

- 只有被提出过的值才可以被选定；
- 只有一个值会被选定；
- 进程只有在某个值确实已经被选定时，才会得知该值已被选定。

我们不试图精确规定活性要求（译者注：这意味着我们只需要保证正确性，能让它最终达成一致就行了，对于效率或者后续提及的“活锁”导致的很难才能选出值的问题不关心，工业界会有成熟的方法解决这个问题）。不

过，目标是保证某个被提出的值最终会被选定；并且如果某个值已经被选定，那么进程最终能够得知这个值。

我们让三类 Agent 承担共识算法中的三种角色：Proposer、Acceptor 和 Learner。在一种实现中，单个进程可以扮演多个 Agent，但 Agent 到进程的映射关系不是这里关心的问题。

译者注：我在 *code* 目录下实现了一个 *Basic Paxos MVP*，包含一份 *Server* 代码和一份 *Client* 代码，其中 *Server* 就同时扮演了 *Proposer*、*Acceptor* 和 *Learner* 的角色，*Client* 只是一个独立进程/外部调用方，不属于 *Paxos* 三个角色中的任意一个，可以理解为外部客户端向这些服务器发送请求，*Server* 自己充当 *Proposer*，执行完后将结果返回给客户端，客户端不会是 *Proposer*。*Server* 为 *Client* 提供了以下接口：

```
func (Server) Get() (hasValue bool, value int);  
func (Server) Set(val int) (setSuccess bool, value int);
```

客户端可以 *Get* 查询当前 *Server* 是否已经选定了值，如果已选定就返回 (*true*, 当前选定的值)，否则返回 (*false*, 0)；也可以 *Set* 写入值，但是由于 *Basic Paxos* 只能选出一个值，如果值已经被选定，那么写入会失败，返回 (*false*, 当前选定的值)。

假设 Agent 之间可以通过发送消息来通信。我们采用通常的异步、非拜占庭模型，其中：

- Agent 可以以任意速度运行，可以因停止而失败，也可以重启。由于所有 Agent 都可能在某个值被选定后失败并重启，所以除非失败后重启的 Agent 能够记住某些信息，否则不可能得到一个解。(译者注：这里暗示了使用非易失存储，例如磁盘)
- 消息可能经过任意长时间才送达，可能被复制，也可能丢失，但不会被篡改 (译者注：所有节点都是“诚实”的，不会伪造消息，这就是“非拜占庭”的含义)。

2.2 选定一个值

选定一个值最简单的办法是只有一个 Acceptor。Proposer 向 Acceptor 发送一个 Proposal，而 Acceptor 选择它收到的第一个被提出的值。这个方案虽然简单，但并不令人满意，因为一旦这个 Acceptor 失败，系统就无法继续取得进展。(译者注：我们希望系统整体具有一定程度的容错能力，部分节点失败不影响整体系统的运行；但超过一定数量的节点失败后，系统无法继续运行也是可以接受的。很显然，我们不希望单个节点失败就让整个系统无法工作，单点故障是无法接受的)

所以，我们试试另一种选定值的办法：使用多个 Acceptor。Proposer 把一个被提出的值发送给一组 Acceptor。Acceptor 可以接受这个被提出的值。当足够大的一组 Acceptor 已经接受它时，这个值就被选定了。多大才算足够大？为了保证只有一个值会被选定，我们可以让“足够大的一组”指任意多数派 Acceptor (译者注：在本论文，*Acceptor* 数量是不会改变的，文章不讨论成员变更的情况，所以多数派的定义是固定的。例如，在总数为 3 个的情况下，多数派就是 2 个或者 3 个。在总数为 5 个的情况下，多数派就是 3/4/5 个)。因为任意两个多数派至少有一个 Acceptor 相交，所以只要每个 Acceptor 至多接受一个值，这个办法就能成立。(对“多数派”概念存在一种显然的推广，这种推广已在大量论文中被讨论，显然最早可追溯到文献 [3]。)

在没有失败和消息丢失的情况下，即使只有一个 Proposer 提出一个值，我们也希望有值被选定 (译者注：这就是前面提到的，论文整体的行文思路是“通过提出一些我们需要的特性来推导算法流程”，这显然是我们希望满足的第一个特性)。这给出了下面的要求：

P1. Acceptor 必须接受它收到的第一个 Proposal。

但这个要求带来了一个问题。多个不同的 Proposer 可能几乎同时提出多个值，导致每个 Acceptor 都已经接受了一个值，但没有任何一个值被多数 Acceptor 接受。即使只有两个被提出的值，如果每个值都被大约一半 Acceptor 接受，那么只要一个 Acceptor 失败，就可能无法知道到底哪

个值被选定。(译者注：这里其实作者有两个假设，一个是 *P1*——也就是 *Acceptor* 必须接受它收到的第一个 *Proposal*；第二个假设是 *Acceptor* 只能接受一个 *Proposal*。举例来说，假如有 4 个 *Acceptor*，两个 *Proposer* 提出了两个不同的值，导致每个 *value* 都被不同的两个 *Acceptor* 接受，那么没有一个 *value* 被多数派接受。这时，值就不能被选出来，直接流程死锁)

P1 以及“只有当一个值被多数 *Acceptor* 接受时才被选定”这个要求意味着：必须允许 *Acceptor* 接受不止一个 *Proposal*(译者注：经过之前的讨论，假设只允许 *Acceptor* 接受一个 *Proposal*，就很有平分票导致流程死锁，那么只能允许接受多个了)。我们给每个 *Proposal* 分配一个自然数编号，用来区分 *Acceptor* 可能接受的不同 *Proposal*。因此，一个 *Proposal* 由 *Proposal* 编号和值组成(译者注：后续把它们称为 *Proposal number* 和 *Proposal value*)。为了避免混淆，我们要求不同 *Proposal*(译者注：什么叫不同的 *Proposal*? 这里很微妙，这里可以暂且理解为“全局/全世界维度”只会有一份 $(\textit{Proposal number}, \textit{Proposal value})$ tuple 对，且不会存在两个 *Proposal number* 相同的 *Proposal*。在这种全局 *Proposal number* 唯一的情况下，一个 *Proposal number* 只对应一个 *value*。*Proposal number* 就是 *Proposal* 的“唯一身份证”)具有不同的 *Proposal number*。如何做到这一点(译者注：这里指的是如何避免不同 *Proposer* 生成相同的 *Proposal* 编号。这个保证由具体实现负责，具体操作可以看 2.5 章节“实现”的最后一段)取决于具体实现，所以现在我们先直接假设它成立。当某个带有值 v 的 *Proposal* 被多数 *Acceptor* *accept*(接受)时，这个值就被“选定”。在这种情况下，我们说这个 *Proposal*(以及它的值)已经被 *chosen*。(译者注：我举个例子，假如一个 *Proposal* 是 $(5, 3)$ ，被大多数 *Acceptor* *accept* 了，那么就说 *Proposal number*=5, *value*=3 的 *Proposal* 被 *chosen* 了，并且 *value*=3 也被 *chosen* 了。“*chosen*”这个词很重要，后续我会频繁地使用“*chosen*”)

我们可以允许多个 *Proposal* 被 *chosen*，但必须保证所有被 *chosen* 的 *Proposal* 都有相同的值。按 *Proposal* 编号归纳，只要保证下面这一点就足够：

P2. 如果一个值为 v 的 *Proposal* 被选定，那么每个编号更大的、

被选定的 Proposal 都具有值 v 。(译者注：这里很好理解，假如 Agent 1 提出 *Proposal*(5,3)，且被全部 *Acceptor* *accept* 了，那么事实上该 *Proposal* 以及对应的 *value* 就被 *chosen* 了；假如随后 Agent 2 提出 *Proposal*(6,4)，也被全部 *Acceptor* *accept* 了，那么其实就“覆写了”之前的工作。为了避免这样的事情发生，就提出了 P2 这个要求：如果一个 *value* 在事实上存在一个时刻被大多数 *Acceptor* *accept*，那么后续就绝对不能出现一个时刻，使得另外一个 *value* 被大多数 *Acceptor* *accept*。这个要求相当强，它强要求集群中的所有大多数只能认可一个值，而不能“切换到另外一个值，这就是 Paxos 的核心思想)

由于 Proposal number 是全序的 (译者注：意味着任何两个 *Proposal number* 之间都有明确的大小关系；再结合 *Proposal number* “全局唯一”的假设，任何两个不同 *Proposal* 的编号都不会相等。具体怎么实现全局唯一的编号生成，这点会在后文说明)，条件 P2 保证了最关键的安全性质：只有一个值会被 *chosen*。

一个 Proposal 要被 *chosen*，至少必须被一个 *Acceptor* *accept*。所以，我们可以通过满足下面这个条件来满足 P2：

P2a. 如果一个值为 v 的 Proposal 被 *chosen*，那么任何 *Acceptor* *accept* 的每个编号更大的 Proposal 都具有值 v 。(译者注：Paxos 要求一个最多只有一个 *value* 被 *chosen*，实际上说的就是在任何时刻，假如集群中存在一个大多数认可了某个值，那么后续就绝对不能出现另一个时刻，使得另外一个 *value* 被大多数 *Acceptor* 认可，然而这个要求太宽泛，太难描述。所以作者提出了 P2a，它显然更严格，且如果能满足它，那么满足 P2 也是很显然的)

我们仍然要求 P1，以保证会有某个 Proposal 被 *chosen*。由于通信是异步的，某个 Proposal 可能已经被 *chosen*，而某个特定的 *Acceptor* c 从未收到过任何 Proposal。假设一个新的 Proposer “醒来”，并发出一个编号更

大、但值不同的 Proposal。P1 要求 c 接受这个 Proposal，从而违反 P2a。要同时保持 P1 和 P2a，就必须把 P2a 加强为：

P2b. 如果一个值为 v 的 Proposal 被 chosen, 那么任何 Proposer 发出的每个编号更大的 Proposal 都具有值 v 。(译者注：P2a 是对 Acceptor 作出了要求，要求 *accept* 满足条件。译者认为提出 P2b 的原因有两个，一就是论文中提到的，“突然醒来”的 case, P2a 和 P1 打架的情况；第二个思考我认为可以这样想：Acceptor 在 Paxos 的设计上更倾向于“孤立”的，不感知其它 Acceptor 和全局状态，所以这里从源头，也就是 Proposer 入手，扩展成了 P2b，对 Proposer 发出的 Proposal 作出要求)

因为 Proposal 必须先由 Proposer 发出，才可能被 Acceptor 接受，所以 P2b 蕴含 P2a，而 P2a 又蕴含 P2。(译者注：蕴含关系就是包含关系，P2b 能推出 P2a，P2a 能推出 P2，也就是说只要满足 P2b，之前提到的 P2a 和 P2 也就满足了)

为了弄清楚如何满足 P2b，我们先考虑怎样证明它成立 (译者注：这里的前提是 (m, v) 已经被 chosen，问题是怎样设计协议，才能保证发出 $n > m$ 的 Proposal 时满足 P2b)。我们会假设某个编号为 m 、值为 v 的 Proposal 已经被 chosen，然后证明任何编号为 $n > m$ 的 Proposal 也具有值 v 。为了让证明更容易，我们可以对 n 做归纳；也就是说，在额外假设所有编号位于 $m..(n-1)$ 之间的已发出 Proposal 都具有值 v 的前提下，证明编号为 n 的 Proposal 具有值 v ，其中 $i..j$ 表示从 i 到 j 的所有编号集合。编号为 m 的 Proposal 要被 chosen，必然存在某个由多数 Acceptor 组成的集合 C ，使得 C 中的每个 Acceptor 都 *accept* 了它。把这一点与归纳假设结合起来，“ m 已被 chosen” 这个假设意味着：

C 中的每个 Acceptor 都已经 *accept* 了某个编号位于 $m..(n-1)$ 之间的 Proposal，并且任何 Acceptor *accept* 的、编号位于 $m..(n-1)$ 之间的每个 Proposal 都具有值 v 。

由于任何由多数 Acceptor 组成的集合 S 都至少包含 C 中的一个成员，所以只要确保下面这个不变式一直成立，我们就能推出编号为 n 的 Proposal 具有值 v ：

P2c. 对任意 v 和 n ，如果一个值为 v 、编号为 n 的 Proposal 被发出，那么存在一个由多数 Acceptor 组成的集合 S (译者注：读者可以想象，在发出前的瞬间，*Proposer* 能够凝固时间，原子地拿到一个大多数 *Acceptor* 的状态)，满足以下二者之一：(a) S 中没有 Acceptor accept 过任何编号小于 n 的 Proposal (译者注：这代表在 n 发出前，一定没有 *chosen* 一个值，因为大多数 *Acceptor* 都说自己没有 *accept*；相应的，想想如果 5 台机器中 n 的 *Proposer* 拿到了三个 *Acceptor* 的状态，其中两个的状态是 *accept value1*，另外一个显示没有 *accept* 任何 *Proposal*，那么其实什么也推断不出来：有可能剩下两个机器都没有 *accept*，此时其实集群就没有 *chosen* 任何值；但是也有可能剩下的机器里面也有一个人 *accept* 了 *value1*，此时就说明 *value1* 被 *chosen* 了；还有一个例子，假如 5 台机器，其中 n 的 *Proposer* 拿到了四个 *Acceptor* 的状态，其中三个都说没有 *accept* 过任何 *Proposal*，那么其实就能推断集群还没有 *chosen* 任何值)；或者 (b) 在 S 中所有 Acceptor accept 过的、编号小于 n 的 Proposal 里，编号最高的那个 Proposal 的值是 v 。

因此，只要保持 P2c 这个不变式，我们就可以满足 P2b。

为了保持 P2c 这个不变式，想要发出编号为 n 的 Proposal 的 *Proposer* 必须获知：在某个多数 Acceptor 集合中的每个 Acceptor 那里，编号小于 n 且已经被 accept 或将会被 accept 的 Proposal 里，编号最高的那个是什么 (如果存在的话)。了解已经被 accept 的 Proposal 很容易；预测未来会 accept 什么则很困难。*Proposer* 不去预测未来，而是通过取得承诺来控制未来：让这些 Acceptor 承诺不会再 accept 这样的 Proposal。换句话说，*Proposer* 请求 Acceptor 不要再 accept 任何编号小于 n 的 Proposal。这导出了如下发

出 Proposal 的算法。(译者注：直接询问所有节点，然后拿到一个大多数，再发出 Proposal 是不明智的，因为任何机器都会以任意的速度运行，状态随时会因为其它的 Proposal 更改，在没有全局锁的情况下，可能在你判断完之后，其它机器已经做了一堆其他 *accept*。分布式是风起云涌的，在不使用全局锁的情况下只能放弃预测未来，而是通过取得承诺来控制未来)

1. Proposer 选择一个新的 Proposal 编号 n ，向某个 Acceptor 集合的每个成员发送请求，要求它回复：
 - (a) 一个承诺：以后绝不再接受编号小于 n 的 Proposal；
 - (b) 它已经 *accept* 过的、编号小于 n 的最高编号 Proposal(如果存在的话)。

我把这样的请求称为编号为 n 的 Prepare 请求。

2. 如果 Proposer 收到了多数 Acceptor 给出的所需响应 (译者注：Proposer 向所有 Acceptor 发送 *Prepare*(n)，只要多数派 *say ok*(也就是接受，并做出拒绝小于 n 的请求的承诺，拿到它们的 *Max Promised Number* 和 *Max Accepted Proposal*)，那么就能进入下一步；如果拿不到多数派的 *ok*，或者干脆没有获得多数个 *Response*，此时只能从 *Prepare* 阶段开始重来 (并且为了尽可能成功 *Prepare*，选择大于每个 *Response* 中的 *Max Promised Number* 的 *Proposal Number*)。这里读者可能会有一个常见的疑问：假如 *Prepare* 得到的多数派响应都报告接受过同一个 Proposal，例如 5 个 Acceptor 中有 3 个都报告接受过 $(9, v)$ ，能不能确定 v 已经被 *chosen*？答案是可以。*chosen* 的定义就是同一 Proposal 曾被多数 Acceptor 接受，这些接受事件不要求同时发生；Acceptor 后来接受更高编号 Proposal，也不会抹去它曾接受 $(9, v)$ 的事实。并且根据 Paxos 的安全性，后续被 *chosen* 的更高编号 Proposal 仍然必须具有值 v 。需要注意的是，如果多数响应只表示它们各自接受过某个值 v ，但 Proposal 编号并不相同，就不能仅凭“值相同”断言某一个 Proposal 已经被 *chosen*)，那么它就可以发出编号为 n 、值

为 v 的 Proposal(译者注：也就是接下来会提到的 *accept* 请求)。这里的 v 是这些响应中最高编号 Proposal 的值；如果 Acceptors 都报告没有 accept 过 Proposal(译者注：这个报告来自于 *Prepare* 请求的返回)，那么 v 可以是 Proposer 自己选择的任意值。(译者注：这里常见的做法是提出客户端传入的值；这里还有一个微妙的点，它强调了“都没有”(这里指的是返回的多个接受 *Response* 中都没有，不一定要大家全部都接受或者回应，也就是允许部分服务器宕机/丢包/拒绝)accept 过 Proposal 才能选择自己的值，否则就只能选择拿到的响应中最高 Proposal number 对应的值了。这里再讨论一个情况：共 5 台机器，在提出 *Prepare* 时，拿到 4 个接受 *Response*，此时三个都说自己没有 accept 过任何 Proposal，只有一个说自己 accept 了某个 Proposal，那么 Proposer 能不能选择自己的 v 呢？答案是，都行，这种情况下可以推断出提出 Proposal 前还没有选出任何值，所以可以选自己的，这符合安全性要求。但工程上继承会更保守，继承值从概率上来说更容易达成一致，所以更推荐继承)

Proposer 通过向某个 Acceptor 集合 (这个集合不必与响应初始请求的 Acceptor 集合相同) 发送请求 (译者注：事实上最好是向所有的 Acceptors 发送请求)，请求它们 accept 这个 Proposal，来发出 Proposal。我们把这种请求称为 accept 请求。

上面描述的是 Proposer 的算法。那么 Acceptor 呢？Acceptor 可能从 Proposer 收到两类请求：*Prepare* 请求和 *Accept* 请求。Acceptor 可以忽略任何请求而不破坏安全性 (译者注：这里说的是安全性：丢包、重复和乱序不会导致选出两个不同的值。但这并不意味着不断重试就必然成功；活性还需要最终能够通信的多数派，以及最终稳定的指定 Proposer 或其他避免 Proposer 持续竞争的机制)。所以，我们只需要说明它何时允许响应某个请求。它总是可以响应 *Prepare* 请求。它可以响应 accept 请求并 accept 该 Proposal，当且仅当它没有承诺不这样做。换句话说：

P1a. Acceptor 可以 accept 编号为 n 的 Proposal，当且仅当它

尚未响应过任何编号大于 n 的 Prepare 请求。(译者注: *Acceptor* 响应 *Prepare(N)* 后会做出承诺: 绝不再接受编号小于 N 的 *Proposal*。对于编号更小的 *Prepare* 请求, *Acceptor* 可以忽略或明确拒绝 (本论文只提到了忽略), 但不能把 *Prepare* 请求本身表述成被 *accept* 的 *Proposal*。值的留意的是, 在本论文中 *Acceptor* 的“响应”其实指的就是 *accept*, 也就是接受; 而忽略则代表拒绝, 但在工业实现中, 拒绝其实也可以直接明确地返回 *Response*, 并且带上 *Max Promised Number*, 帮助 *Proposer* 跳号, 这些代码都可以在 *code* 里面看到)

注意, P1a 包含了 P1。

现在我们已经有了一个完整的、满足所需安全性质性质的选值算法, 前提是 *Proposal* 编号唯一。最终算法还要加入一个小优化。

假设某个 *Acceptor* 收到编号为 n 的 *Prepare* 请求, 但它已经响应过编号大于 n 的 *Prepare* 请求, 因此已经承诺不会 *accept* 任何新的编号为 n 的 *Proposal*。此时这个 *Acceptor* 没有理由响应新的 *Prepare* 请求, 因为它不会 *accept* 该 *Proposer* 想要发出的编号为 n 的 *Proposal*。所以, 我们让 *Acceptor* 忽略这样的 *Prepare* 请求。我们也让它忽略针对它已经 *accept* 过的 *Proposal* 的 *Prepare* 请求。(译者注: 我不太认可响应这个词, 理论上 *Acceptor* 可以明确接受或者拒绝这个 *Prepare*, 在论文作者的语义中, 它认为忽略就是拒绝, 响应就是接受并承诺。而我认为我们不应该忽略, 而是通过一个 *bool* 控制明确接受或拒绝, 拒绝的时候可以带出 *Acceptor* 当前的 *Max Promised Number*, 帮助 *Proposer* 更快地跳号以达成目标)

有了这个优化, *Acceptor* 只需要记住它曾经 *accept* 过的最高编号 *Proposal*, 以及它已经响应过的最高编号 *Prepare* 请求的编号 (译者注: *Max Proposal* 和 *Max Promised Number*)。由于无论是否发生失败, P2c 都必须保持为不变式, 所以即使 *Acceptor* 失败后重启, 也必须记住这些信息。注意, *Proposer* 总是可以放弃一个 *Proposal*, 并完全忘记它, 只要它绝不再尝试发出另一个具有相同编号的 *Proposal* 即可。

把 *Proposer* 和 *Acceptor* 的动作合在一起, 可以看到算法分两阶段运

行。

阶段 1。

(a) Proposer 选择一个 Proposal 编号 n ，向多数 Acceptor 发送编号为 n 的 Prepare 请求。

(b) 如果 Acceptor 收到编号为 n 的 Prepare 请求，且 n 大于它已经响应过的任何 Prepare 请求编号，那么它响应 (译者注：接受并承诺) 这个请求：承诺不再 accept 任何编号小于 n 的 Proposal，并附上它已经接受过的最高编号 Proposal (如果存在的话)。译者注：实际上论文这里遇到 n 小于 Acceptor 的情况时选择不响应，但是工程上更合理的返回如下：

```
message ProposalNumber {
    int64 round = 1;
    int64 machine_id = 2;
}

message PrepareResponse {
    bool ok = 1; // true=Promised, false=Reject
    // 见过的最大Promised编号，Reject时供Proposer跳号
    ProposalNumber max_promised_number = 2;
    // 历史上接受过的最高编号提案
    ProposalNumber max_accepted_number = 3;
    bytes max_accepted_value = 4;
}
```

阶段 2。

(a) 如果 Proposer 收到多数 Acceptor 对其编号为 n 的 Prepare 请求的响应，那么它向这些 Acceptor 中的每一个发送 accept 请求，请求它们接受编号为 n 、值为 v 的 Proposal。这里的 v 是所有响应中最高编号 Proposal 的值；如果响应报告没有 Proposal，那么 v 可以是任意值。译者注：这里提到，至少拿到多数 Acceptor 的接受响应，也就是拿到多个 $ok==true$ 的 Prepare Response 才能进行阶段二 accept 请求。并且 accept 选择的 value 必须是这些 Response 中 Proposal number 最大的那个 Proposal 的 value。但参

见我们之前的讨论，假如集群 5 个 *Peers*，*Prepare* 拿到 4 个 *ok==true*，其中三个说自己此前没有 *accept* 任何 *Proposal*，那么即使剩下那个 *Response* 显示有 *Proposal* 被接受，其实也可以忽略它并提出自己的值，这在安全性上是说的过去的，相当于“意外地”丢掉了有一个 *Response*，且拿到的三个 *Response* 都说自己没有 *accept* 过任何 *Proposal*，由于这仍然是一个大多数，所以这是安全的。但是一般没人这么干，因为继承旧值显然更有效率，更容易选出值。下面是 *accept* 的响应定义：

```
// 以 Protobuf 为例的工程化定义
message AcceptResponse {
    bool ok = 1;
    ProposalNumber max_promised_number = 2;
}
```

(b) Acceptor 收到编号为 n 的 *accept* 请求时，检查自己承诺过的最大编号：如果这个编号不大于 n (译者注：*Prepare* 的接受条件是 $n >$ 节点记录的承诺值，而 *Accept* 的接受条件是 $n \geq$ 节点记录的承诺值，这两个条件有微小差异，一个有等号，一个没有，读者请注意！)，就接受该 *Proposal*；否则拒绝。(译者注：也就是当 $n \geq MaxPromisedNumber$ 时返回 *ok==true*，并将 *MaxPromisedNumber* 更新为 n ；否则返回 *ok==false* 和当前的 *MaxPromisedNumber*，*Proposer* 可以据此换用更大的编号重试。一个 *value* 被多数 *Acceptor* 接受后，就被选定了。这个 *value* 可能是 *Proposer* 自己提出的，也可能是它在 *Prepare* 阶段继承的旧值。因此，客户端调用 *Set* 后，服务端应返回最终选定的值。)

只要对每个 *Proposal* 都遵循这个算法，一个 *Proposer* 可以发出多个 *Proposal*。它也可以在协议中途的任何时刻放弃某个 *Proposal* (即使该 *Proposal* 的请求和/或响应在它被放弃很久之后才到达目的地，正确性仍然保持)。如果某个 *Proposer* 已经开始尝试发出编号更大的 *Proposal*，那么放弃当前 *Proposal* 很可能是个好主意。因此，如果 *Acceptor* 因为已经收到过编号更高的 *Prepare* 请求而忽略某个 *Prepare* 或 *accept* 请求，那么它大概应该告知 *Proposer*；*Proposer* 随后应该放弃自己的 *Proposal*。这是一种不影

响正确性的性能优化。(译者注：这里在说一个优化，意思是不论是 *Prepare* 还是 *Accept* 请求，如果在 *Response* 里发现 *Max Promised Number* 大于自己的 *Proposal Number* 时，说明该放弃，直接重开了，自己的 *Proposal* 一定选不上了)

2.3 获知被选定的值

为了得知某个值已经被选定，Learner 必须知道某个 Proposal 已经被多数 Acceptor 接受。显然的算法是：每当 Acceptor 接受某个 Proposal 时，就向所有 Learner 回复并发送这个 Proposal。这样 Learner 可以尽快知道被选定的值，但它要求每个 Acceptor 都向每个 Learner 回复，响应数量等于 Acceptor 数量与 Learner 数量的乘积。

非拜占庭失败这个假设使得一个 Learner 很容易从另一个 Learner 那里得知某个值已经被接受。我们可以让 Acceptor 把自己的 Acceptance 发送给一个指定 Learner，再由这个指定 Learner 在某个值被选定时通知其他 Learner。这种做法需要额外一轮通信，所有 Learner 才能发现被选定的值。它的可靠性也较差，因为指定 Learner 可能失败。但它需要的响应数量只等于 Acceptor 数量和 Learner 数量之和。

更一般地，Acceptor 可将 Acceptance 发送给一组指定的 Learner。每个指定 Learner 都可以在某个值被选定时通知所有 Learner。增加指定 Learner 的数量可以提高可靠性，代价是更高的通信复杂度。

译者注：上面说的是 *Learner* 和 *Acceptor* 是两组集群，其中客户端可以向 *Learner* 询问最终选择的值，这里谈了怎么让 *Learner* 知道最终被 *chosen* 的值。上面提到的做法是把 *Acceptance* (也就是某个 *Instance* 接受了某个 *Proposal* 的信息) 发送给 *Learner*。*Learner* 可以在内存里面维护一张表，*Key* 为 *ProposalNum*，*Value* 为已收到的 *Acceptor* 集合，当某个 *Proposal* 的 *Acceptor* 集合大小达到多数，就可以肯定这个值被 *chosen* 了 (这个可能比较难以理解，读者可以多想想具体的 *case*)。关于“发送”，主要讨论的是解决 *Fan Out* 问题的方案，其中提到了三个方法，一个是 *Acceptor* 接受一

个 *accept* 请求时发送给所有 *Learner* 这个信息，这是全量广播，很容易引发广播风暴；第二个方法是指定一个 *Learner*，所有 *Acceptor* 只向它发送 *Acceptance*，等它确定某个 *Proposal* 被多数 *Acceptor* 接受后，就向所有其它 *Learner* 广播信息，这个方法的缺点是容易引发单点故障；第三种方法是综合了第一种和第二种，指定一组 *Learner* 接收 *Acceptance*，其中任意节点得知某个 *Proposal* 获得多数 *accept* 后，就向其它所有 *Learner* 广播这个信息。当客户端向某个 *Learner* 询问是否选出行，即使集群中实际已经 *chosen*，如果 *Learner* 尚未收到足够广播或攒够多数派，也无法立刻确认，因此 *Learner* 的查询具有滞后性，要解决这个问题，可以看下面的文本

由于消息可能丢失，一个值可能已经被选定，却没有任何 *Learner* 知道。*Learner* 可以询问 *Acceptor* 它们接受过哪些 *Proposal*，但如果某个 *Acceptor* 失败，就可能无法知道某个特定 *Proposal* 是否已经被多数 *Acceptor* 接受。在这种情况下，*Learner* 只有当一个新的 *Proposal* 被选定时，才会知道被选定的值是什么。如果某个 *Learner* 需要知道是否已经有值被选定，它可以让一个 *Proposer* 使用前面描述的算法发出一个 *Proposal*。(译者注：*Learner* 可以让一个 *Proposer* 发起 *Prepare*，并收集多数派 *Acceptor* 的响应。如果多数派均表示从未接受过任何 *Proposal*，那么根据多数派交集性质，可以确定在本次 *Prepare* 取得多数派 *Promise* 的时刻之前，尚未有值被 *chosen*，因此可以返回 *Null*。否则，*Prepare* 响应中出现的 *value* 不一定已经被 *chosen*，*Proposer* 需要按照 *Paxos* 规则，选择响应中编号最高的已接受 *Proposal* 所对应的 *Value*，并继续完成 *Accept* 阶段，使该 *value* 最终被 *chosen*。需要注意，这种读取可能会主动选定一个此前尚未 *chosen* 的值，因此它不仅是查询，也可能修改 *Paxos* 实例的状态。由于读取通过多数派建立了一个线性化点，所以不会返回滞后的结果，可以视为线性一致性读)

2.4 进展

很容易构造这样一种场景：两个 *Proposer* 各自不断发出一系列编号递增的 *Proposal*，但没有任何 *Proposal* 被选定。*Proposer p* 完成了编号为 n_1

的 Proposal 的阶段 1。另一个 Proposer q 随后完成了编号为 $n_2 > n_1$ 的 Proposal 的阶段 1。由于 Acceptor 都已经承诺不再接受任何编号小于 n_2 的新 Proposal，所以 Proposer p 针对编号为 n_1 的 Proposal 发出的阶段 2 accept 请求会被忽略。于是 Proposer p 又开始并完成编号为 $n_3 > n_2$ 的新 Proposal 的阶段 1，导致 Proposer q 第二阶段的 accept 请求被忽略。如此往复。

为了保证进展，必须选择一个指定 Proposer，只有它尝试发出 Proposal。如果指定 Proposer 能够与多数 Acceptor 成功通信，并且使用的 Proposal 编号大于所有已使用编号，那么它将成功发出一个会被接受的 Proposal。如果它得知某个请求带有更高 Proposal 编号，就放弃自己的 Proposal 并重试，那么这个指定 Proposer 最终会选择一个足够高的 Proposal 编号。

因此，如果系统中足够多的部分 (Proposer、Acceptor 和通信网络) 正常工作，就可以通过选出单个指定 Proposer 来实现活性。Fischer、Lynch 和 Paterson 的著名结果 [1] 说明，用来选举 Proposer 的可靠算法必须使用随机性或真实时间，例如使用超时。不过，无论选举成功还是失败，安全性都能得到保证。

2.5 实现

Paxos 算法 [5] 假设存在一个由进程组成的网络。在它的共识算法中，每个进程都扮演 Proposer、Acceptor 和 Learner 的角色 (译者注：可以把这些角色称为 *Peers*)。算法会选择一个 Leader，由它扮演指定 Proposer 和指定 Learner (译者注：这里对应了之前我提到的算法活性差，容易活锁的一个工业解法，选出单个 *Proposer* 作为 *Leader*，让它作为 *Proposer*，虽然这个做法经常在 *Multi Paxos* 中被提到，但是读者应该明白其实这一段还是在谈 *Basic Paxos*)。Paxos 共识算法正是上面描述的算法，请求和响应作为普通消息发送 (响应消息会带上对应的 Proposal 编号，以免混淆)。失败期间仍能保存的稳定存储用来维护 Acceptor 必须记住的信息。Acceptor 在实际发送响应之前，会先把它打算给出的响应记录到稳定存储中。译者注：*Raft*

的 *Leader* 选举有数据要求, *Raft* 只允许投票给数据比自己新, 或者至少和自己一样新的 *Peer*, 这是 *Raft* 的核心安全性要求。但是 *Paxos* 的选举, 不论谁当选都能保证安全性, 脑裂也最多影响活性, 选举的唯一目的就是避免活锁, 提高效率。并且对于 *Multi Paxos*, 工程上将选举和 *Prepare* 阶段融合, 一次性 *Prepare* 完下一个任期的所有 *Instance*, 这样后续最优只需要一轮 *RPC* 来回就能决策出一个 *Instance* 了, 大大优化了效率, 达到了理论效率的极限。关于 *Basic Paxos* 如何选举, 实际上相当自由, 因为怎么做都是“安全的”, 我认为做一个“*Raft* 选举弱化版”, 加入心跳 *RPC* 和选举 *RPC* 并去掉选举的安全性要求 (不对比数据, 只对比 *Term*)。具体的思路是做一个随机超时定时器, 当超时后成为 *Candidate*, 并且投自己一票, 然后向所有 *Peers* 发送 *ReqVote(newTerm)* 的 *RPC*。规则是: 1. 任何请求都带有 *Term*, 当 *Peer* 收到任何 *Term* 大于自己时, 立马退化为普通 *Peer*, 并且把自己的 *Term* 更新为请求的 *Term*, 并进行处理; 2. 当遇到 *Term* < 自己的 *Term* 时, 直接忽略请求; 3. 当处理 *ReqVote* 请求时, 每一个 *Term* 只允许投给最先到达的 *Candidate* 一票, 并且发送 *RPC Response* 前先持久化; 4. 当 *Candidate* 收到多数节点的投票时, 成为 *Leader*, 此时开启心跳定时器, 定时发送心跳 *RPC* 抑制其他 *Peer* 成为 *Candidate*; 5. *Client* 只能找到 *Leader* 来进行 *Prepare+Accept* 操作, 或者发送给别的节点, 让节点作为代理转发给 *Leader*, 让 *Leader* 进行 *Prepare+Accept* 操作

剩下要描述的只有一种机制: 如何保证永远不会有两个 *Proposal* 以相同编号被发出 (译者注: *Basic Paxos* 不允许不同的 *Proposer* 提出相同的 *Proposal Number*)。不同 *Proposer* 从彼此不相交的编号集合中选择编号, 所以两个不同的 *Proposer* 绝不会发出编号相同的 *Proposal* (译者注: 解决不同 *Proposal* 的 *Proposal number* 冲突, 实际上很常见的做法是使用 (递增号, 节点标识) 这样的二元组作为 *ProposalNumber*, 我在 *code* 中实现的代码也是这样, 排序规则是先按递增号排序, 若相同再按节点标识排序)。每个 *Proposer* 会在稳定存储 (译者注: 如磁盘) 中记住它曾经尝试发出的最高编号 *Proposal*, 并在阶段 1 中使用一个高于自己已经用过的所有编号的 *Proposal* 编号 (译者注: 持久化是用来解决崩溃后恢复的)。

3 实现状态机

译者注：这一章专注讨论 *Multi Paxos*。前面通篇讲的都是 *Basic Paxos*，它的职责很专一，只能确定一个值，用处不大。这里简单介绍了 *Multi Paxos* 状态机，可以将它理解为：如何使用多台机器，结合 *Paxos* 共识算法，实现一个强一致、可容灾、对外表现得像单机的分布式系统。只要仍有一个内部可以相互通信的多数派，这个系统就能在少于半数服务器宕机或失联时继续取得进展；发生网络分区时，只有包含多数派的一侧能够继续提交，少数派一侧不能提交。简而言之，*Multi Paxos* 基本上就是在用成本换可用性。

实现分布式系统的一种简单方法是：让一组客户端向一个中央服务器发出命令。这个服务器可以被描述为一个确定性状态机（译者注：这个名字在 *VMware Fault-Tolerant* 亦有记载，思路很简单，就是两个相同的状态机，输入同样的命令，就会具有同样的状态，产生同样的输出；但是读者可以留意一下读取时间或者 *CPU Cycle* 这些非确定性命令，这些命令则不能用于确定性状态机，解法 *VMware Fault-Tolerant* 有提到，就是 *Master* 来执行，*Slave* 负责抄答案就行了；我们作为 *Paxos* 的使用者，一般遇到的都是确定性命令，例如实现一个简单的 *KV* 数据库，或者分布式锁等等），它按照某个顺序执行客户端命令。状态机具有当前状态；它通过接收一个命令作为输入，并产生一个输出和一个新状态来执行一步。例如，一个分布式银行系统的客户端可能是柜员，而状态机状态可能包含所有用户的账户余额。取款操作可以通过执行一条状态机命令来完成：当且仅当账户余额大于取款金额时减少该账户余额，并把旧余额和新余额作为输出。

使用单个中央服务器的实现会在该服务器失败时失效（译者注：单机服务虽然是强一致的，但是也是容易单点故障的，可用性低）。因此，我们改为使用一组服务器，每个服务器都独立实现该状态机。由于状态机是确定性的，如果所有服务器都执行同一串命令，它们就会产生相同的状态序列和输出序列。发出命令的客户端随后可以使用任意服务器为它生成的输出。

为了保证所有服务器都执行同一串状态机命令，我们实现一系列彼此独立的 *Paxos* 共识算法 Instance（译者注：*Instance* 又被很多人称为 *Slot*。每

一个 *Instance/Slot* 代表一个 *Basic Paxos*, 决定一条命令; 多个 *Slot* 就组成多条命令, 形成了系统的一连串输入, 也就是命令), 其中第 i 个 *Instance* 选定的值就是命令序列中的第 i 条状态机命令。每个服务器在算法的每个 *Instance* 中都扮演所有角色 (*Proposer*、*Acceptor* 和 *Learner*)。现在, 我先假设服务器集合是固定的, 因此所有共识算法 *Instance* 都使用同一组 *Agent* 集合 (译者注: 论文只讨论固定数量的 *Peers*, 大家互相知道彼此, 并且成员不会变更, 每一个 *Instance* 都使用同样的 *Peers* 集合)。

在正常运行情况下, 系统会选举出一台服务器作为 *Leader*。它会在所有的共识算法实例中充当“独占的主 *Proposer*” (即唯一有权发起提案的 *Proposer*) (译者注: *Leader* 是为了解决活锁问题的。额外地, 在 *Multi Paxos* 中, 选举被融入到了 *Prepare* 中, 因此还可以批量 *Prepare* 一个任期内的全部 *Proposal*, 后续一个命令只需要一个 *RTT* 来回就能确定一个 *Instance* 了, 达到理论上的最优性能)。客户端把命令发送给 *Leader*, 由 *Leader* 决定每条命令应该出现在序列中的哪个位置。如果 *Leader* 决定某条客户端命令应该成为第 135 条命令, 它就尝试让这条命令成为第 135 个共识算法 *Instance* 选定的值。通常它会成功。它可能因服务器或通信故障而失败, 也可能因为另一个服务器也认为自己是 *Leader*, 并且对第 135 条命令应该是什么有不同想法而失败。但共识算法保证, 至多只有一条命令会被选为第 135 条。

这种方法高效的关键在于 (译者注: *Basic Paxos Instance* 需要两轮 *RTT* 确认, 也就是 *Prepare-Accept*, 其中第一轮 *Prepare* 并不需要知道命令本身, 只有第二阶段需要知道; *Multi Paxos* 的 *Leader* 选举利用了这一点, 它将选举融入到了 *Prepare* 中, 并且可以一次性 *Prepare* 一个 *Ballot* 内所有的 *Proposal*。因此, 后续提出命令只需要一次 *Accept* 的 *RTT* 即可, 达到理论最优), 在 *Paxos* 共识算法中, 要提出的值直到阶段 2 才被选择。回想一下, 在完成 *Proposer* 算法的阶段 1 之后, 要么要提出的值已经被确定, 要么 *Proposer* 可以自由提出任意值。

现在我将描述 *Paxos* 状态机 (译者注: *Multi Paxos*) 实现在正常运行期间如何工作。之后再讨论可能出错的情况。我考虑这样一种情形: 前一个

Leader 刚刚失败，新的 Leader 已经被选出。(系统启动是一个特例，此时还没有任何命令被提出)

新的 Leader 作为所有共识算法 Instance 中的 Learner，应该知道大多数已经被选定的命令。假设它知道命令 1-134、138 和 139，也就是共识算法实例 1-134、138 和 139 中被选定的值 (后面会看到命令序列中这样的空洞是怎样产生的)。然后它执行 Instance 135-137 以及所有大于 139 的 Instance 的阶段 1 (下面会说明这是怎么做的)。假设这些执行结果决定了 Instance 135 和 140 中要提出的值，但其他所有 Instance 中的待提议值都没有受到约束。Leader 随后对实例 135 和 140 执行阶段 2，从而选定命令 135 和 140。

此时，Leader 以及任何获知了 Leader 所知全部命令的其他服务器，都可以执行命令 1-135。不过，它还不能执行同样已经知道的命令 138-140，因为命令 136 和 137 尚未被选定。Leader 可以把客户端接下来请求的两条命令作为命令 136 和 137。相反，我们让它立即用一种特殊的 “No-op” 命令填补空洞；这种命令作为命令 136 和 137 被提出，并且不会改变状态 (它通过执行共识算法实例 136 和 137 的阶段 2 来做到这一点)。一旦这些 No-op 命令被选定，命令 138-140 就可以执行了。(译者注：No-op 是一种很常见的命令，它不会对状态机的业务状态产生影响。在 Raft 中，新 Leader 上任后可以追加一条属于当前 Term 的 No-op，并将其复制到多数派，从而推进 *commitIndex*，使它之前尚未提交的日志也随之提交。这里也可以使用一条正常的客户端命令代替 No-op；主动追加 No-op 的好处是不必等待新的客户端请求。Leader 不能仅凭以前 Term 产生的某条日志已经复制到多数派，就直接判定该日志已经提交。读者可以参阅 *In Search of an Understandable Consensus Algorithm* 中的 Figure 8：旧 Term 日志即使一度存在于多数派，仍可能在后续 Leader 选举后被覆盖。Raft 的 Leader 只能根据“当前 Term 的日志已复制到多数派”直接推进 *commitIndex*；一旦当前 Term 的日志提交，它之前的连续日志也会被间接提交。Raft 和 Multi-Paxos 中的 No-op 虽然都不改变状态机的业务状态，但主要用途有所不同：Raft 中的 No-op 主要用于让新 Leader 尽快建立当前 Term 的提交点，并推进此前日志的提交；Multi-Paxos 中的 No-op 主要用于填补尚未 chosen 的 Slot。只有这些

空缺 *Slot* 被确定为 *No-op* 后，状态机才能按照 *Slot* 顺序继续执行后面的命令)

现在，命令 1-140 都被选定。Leader 也已经完成了所有大于 140 的共识算法 Instance 的阶段 1，并且可以在这些 Instance 的阶段 2 中自由提出任意值。它把编号 141 分配给客户端请求的下一条命令，并在共识算法 Instance141 的阶段 2 中把它作为值提出。它把收到的下一条客户端命令作为命令 142 提出，依此类推。

Leader 可以在得知它提出的命令 141 已经被选定之前，就提出命令 142。它在提出命令 141 时发送的所有消息都可能丢失，而命令 142 可能 在其他任何服务器知道 Leader 把什么作为命令 141 提出之前就被选定。当 Leader 没有收到 Instance141 中阶段 2 消息的预期响应时，它会重传这些消息。如果一切顺利，它提出的命令会被选定。不过，它也可能先失败，在已选定命令序列中留下一个空洞。一般来说，假设 Leader 可以领先 α 条命令，也就是说，在命令 1 到 i 已经被选定之后，它可以提出命令 $i + 1$ 到 $i + \alpha$ 。那么就可能产生最多 $\alpha - 1$ 条命令的空洞 (译者注：这里提到限制空洞后 *Leader* 最多能接受的命令数，这个窗口可以用来控制系统的并发性，起限流背压，防止客户端过度请求，导致无限提交提案，超过整个系统的容忍度)。

新选出的 Leader 会为无穷多个共识算法 Instance 执行阶段 1。在上面的场景中，就是 Instance135-137 以及所有大于 139 的 Instance。通过对所有 Instance 使用相同的 Proposal 编号，它可以用一条相当短的消息把这件事发送给其他服务器。在阶段 1 中，Acceptor 只有在已经从某个 Proposer 那里收到过阶段 2 消息时，才会回复比简单 OK 更多的信息 (在这个场景中，只有 Instance135 和 140 属于这种情况)。因此，作为 Acceptor 的服务器可以用一条相当短的消息回复所有实例。所以，为无穷多个 Instance 执行阶段 1 并不会造成问题。

由于 Leader 失败和新 Leader 选举应该是少见事件，执行一条状态机命令的有效成本，也就是对该命令/值达成共识的有效成本，就是只执行共识算法阶段 2 的成本。可以证明，在存在故障的情况下，Paxos 共识算法的

阶段 2 具有所有达成一致算法中可能的最低成本 [2]。因此，Paxos 算法本质上是最优的。

上述关于系统正常运行的讨论假设系统总是只有一个 Leader，除了当前 Leader 失败和新 Leader 选举之间的一小段时间。在异常情况下，Leader 选举可能失败。如果没有服务器作为 Leader，那么就不会有新命令被提出。如果多个服务器都认为自己是 Leader，那么它们可能在同一个共识算法 Instance 中都提出值，这可能阻止任何值被选定。不过，安全性仍然保持：两个不同服务器绝不会对第 i 条状态机命令选定的值产生分歧。选出单个 Leader 只是为了保证进展 (译者注：由于网络分区导致的脑裂问题不会影响安全性，Paxos 对 Leader 的要求相当宽松，因为 Instance 本身要满足 Basic Paxos 才能提交，最坏的情况也就是选不出任何 Instance 的值，但是也不会破坏一致性)。

如果服务器集合可以变化，那么必须有办法确定哪些服务器实现哪些共识算法 Instance。最简单的办法是通过状态机本身来完成。当前服务器集合可以成为状态的一部分，并且可以通过普通状态机命令来改变。我们可以允许 Leader 领先 α 条命令，方式是让执行共识算法 Instance $i + \alpha$ 的服务器集合由第 i 条状态机命令执行之后的状态来指定。这允许我们简单地实现任意复杂的重配置算法。(译者注：前文使用 α 作为 Leader 的流水线窗口，限制 Leader 最多能够并行推进多少个尚未形成连续选定前缀的 Instance。该限制能够约束在途共识实例的数量和资源占用，并形成一定的背压，但它不是直接针对客户端请求的限流机制。在成员变更方面， α 还有一个关键作用：成员变更可以作为普通状态机命令，由某个 Instance 决定。若第 i 条命令执行后改变了状态中的服务器集合，则新集合不立即生效，而是用于执行 Instance $i + \alpha$ 。由于 Leader 最多只能领先 α 条命令，当它启动 Instance $i + \alpha$ 时，第 i 条命令及其产生的配置已经确定。因此，所有服务器都可以根据已经选定的历史状态，确定每个 Instance 应由哪个服务器集合执行，避免对成员配置产生歧义)

参考文献

- [1] Michael J. Fischer, Nancy Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, April 1985.
- [2] Idit Keidar and Sergio Rajsbaum. On the cost of fault-tolerant consensus when there are no faults—a tutorial. Technical Report MIT-LCS-TR-821, Laboratory for Computer Science, Massachusetts Institute Technology, Cambridge, MA, 02139, May 2001. Also published in *SIGACT News* 32(2), June 2001.
- [3] Leslie Lamport. The implementation of reliable distributed multiprocess systems. *Computer Networks*, 2:95–114, 1978.
- [4] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, July 1978.
- [5] Leslie Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, May 1998.